Advanced SQL

Q1. What is a Common Table Expression (CTE), and how does it improve SQL query readability?

- Definition: A CTE (defined using the `WITH` clause) is a temporary result set defined within the execution scope of a single `SELECT`, `INSERT`, `UPDATE`, or `DELETE` statement.
- Readability Improvements:
  Modular Structure: It breaks down complex, deeply nested subqueries into logical, sequential steps.
  Reusability: You can reference the named CTE multiple times within the main query.
  Elimination of Derived Tables: It avoids confusing inline subqueries in `FROM` clauses, making code much easier to maintain and debug.

Q2. Why are some views updatable while others are read-only? Explain with an example.

- Updatable Views: A view is updatable if the database management system (DBMS) can establish a direct, unambiguous one-to-one mapping between the rows/columns in the view and the underlying base table. This generally requires a single table with no aggregations, `GROUP BY`, or `DISTINCT`.
- Read-Only Views: Views that involve groupings, joins, or aggregate functions (`SUM`, `AVG`, `COUNT`) are read-only because the database cannot reliably translate changes made to an aggregated view back into individual rows of the source tables.
- Example:
  Updatable: `CREATE VIEW vw_Products AS SELECT ProductID, Price FROM Products;` (Updating a price here directly maps back to the `Products` table).
  Read-Only: `CREATE VIEW vw_AvgPrice AS SELECT Category, AVG(Price) FROM Products GROUP BY Category;` (Updating an average price is ambiguous because the database doesn't know which individual product prices to adjust).

Q3. What advantages do stored procedures offer compared to writing raw SQL queries repeatedly?

- Performance & Caching: The database engine compiles and optimizes the execution plan of a stored procedure the first time it runs, caching it for subsequent calls.
- Reduced Network Traffic: Applications only need to send the procedure name and parameters instead of lengthy SQL text across the network.
- Security: You can restrict direct table access and grant users permissions *only* to execute specific stored procedures.

- Maintainability: Business logic is stored centrally in the database rather than being scattered across multiple client applications.

Q4. What is the purpose of triggers in a database? Mention one use case where a trigger is essential.

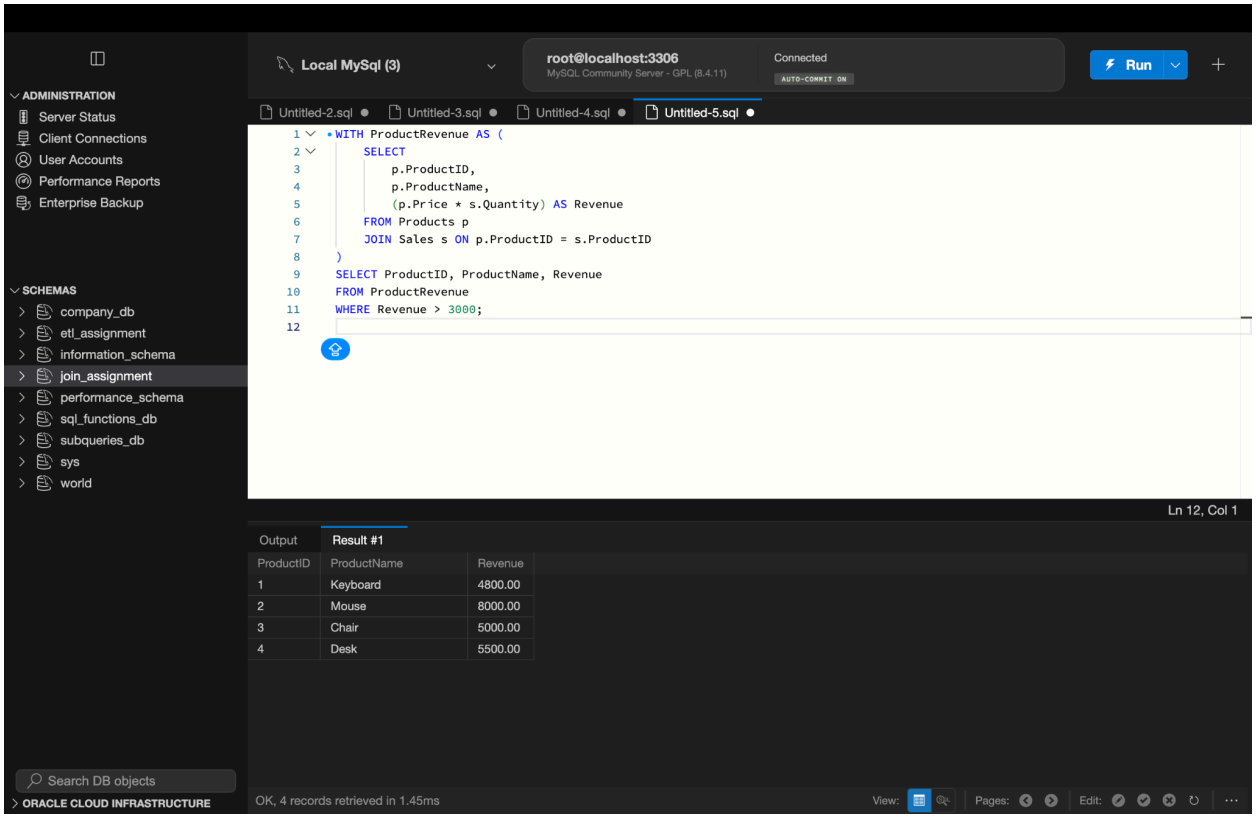
- Purpose: A trigger is a stored procedure that automatically fires (executes) in response to specific database events (such as `INSERT`, `UPDATE`, or `DELETE`) on a table. They enforce complex business rules and data consistency automatically.
- Essential Use Case (Audit Logging / Archiving): Whenever sensitive records (like financial transactions or user deletions) occur, a trigger can automatically capture the deleted or old data and insert it into an archive table with timestamps for compliance and auditing.

Q5. Explain the need for data modelling and normalization when designing a database.

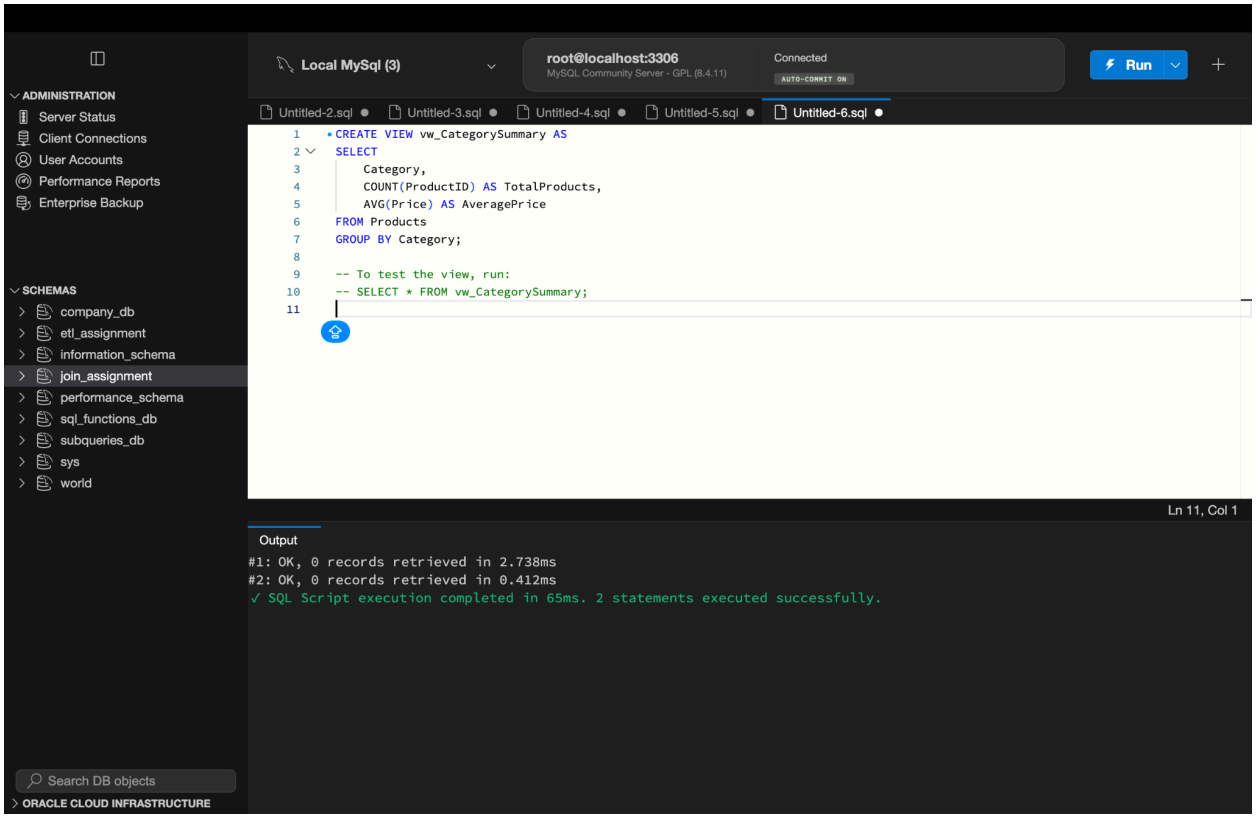
- Data Modelling: Creates a blueprint of data structures and relationships, ensuring the database meets application needs before coding begins.
- Normalization (1NF, 2NF, 3NF, etc.): The process of organizing data to eliminate redundancy and dependencies.
- Benefits:
  Reduces Redundancy: Saves storage space.
  Prevents Anomalies: Avoids update, insertion, and deletion anomalies that lead to data corruption.

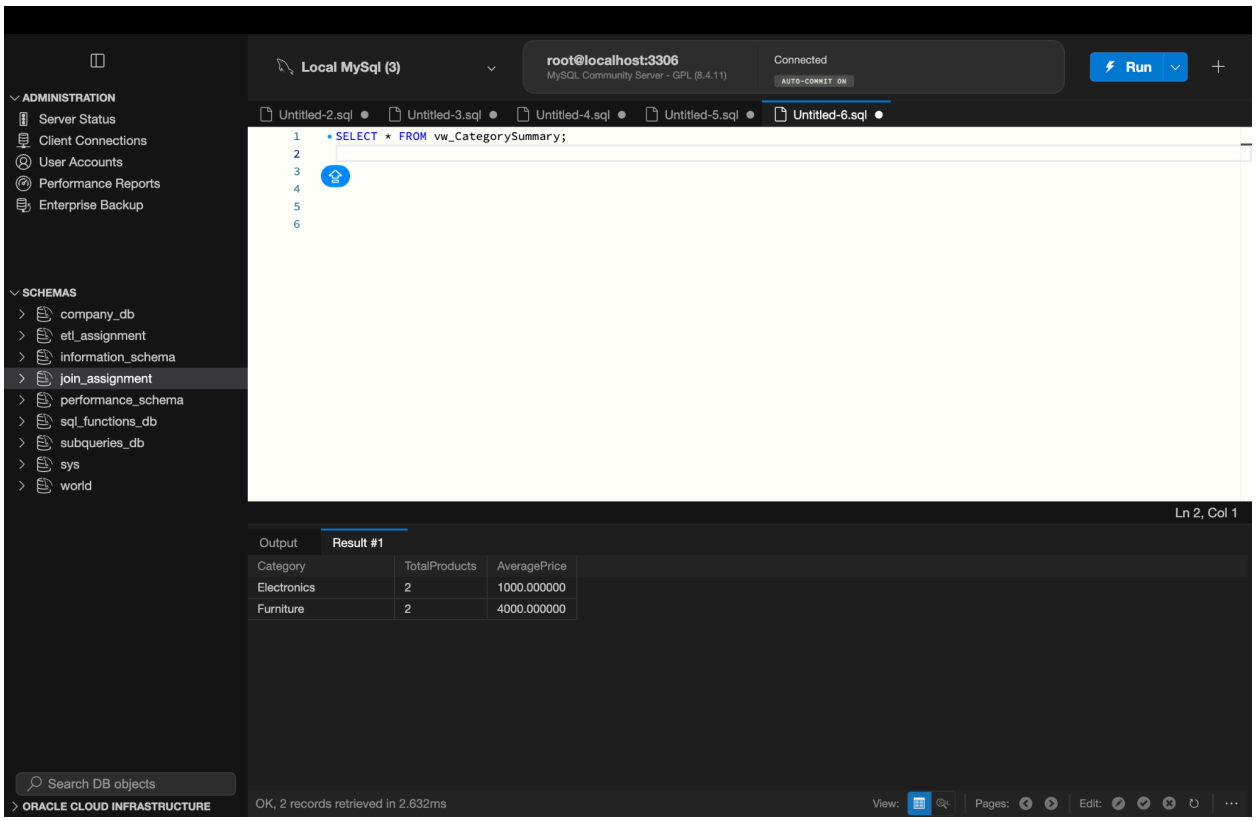
  Ensures Integrity: Keeps relationships consistent across tables.

Q6. Write a CTE to calculate the total revenue for each product (`Revenue = Price * Quantity`), and return only products where revenue > 3000.

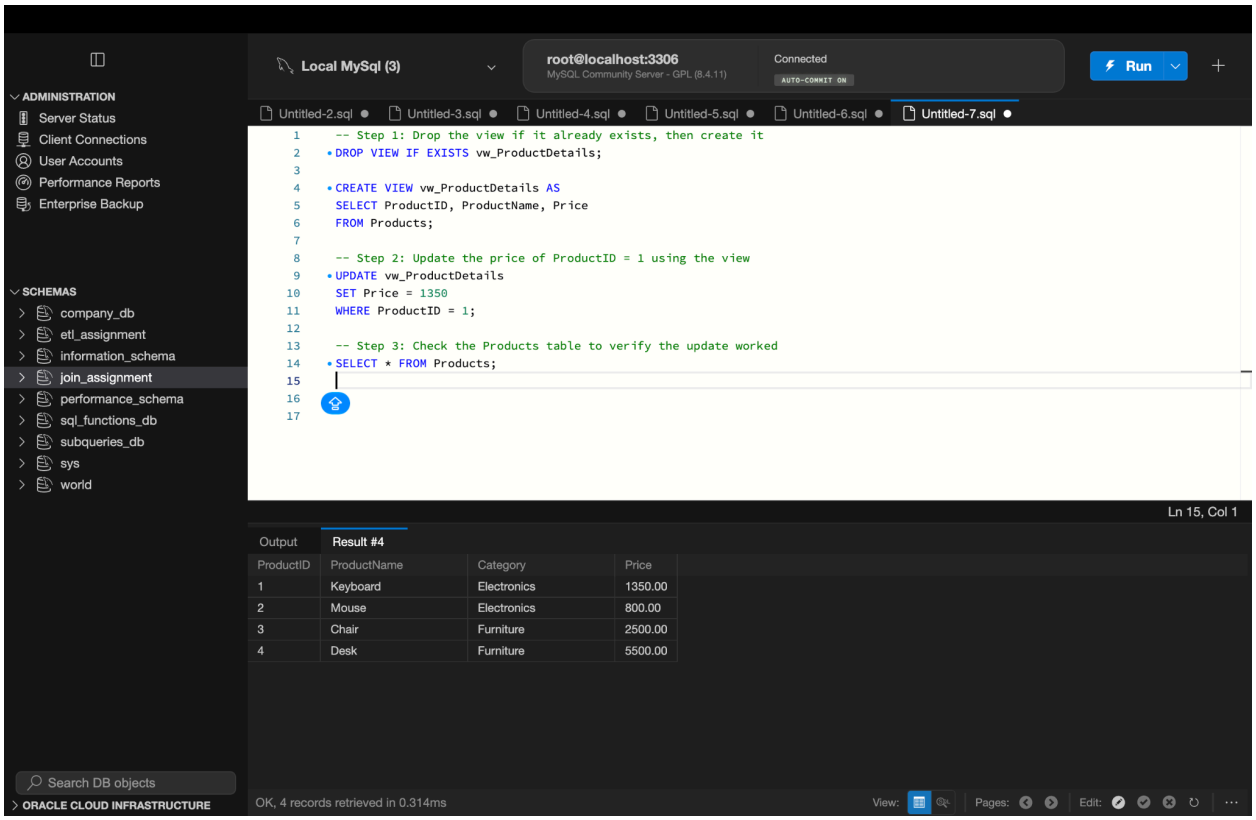


Q7. Create a view named that shows: Category, TotalProducts, AveragePrice.

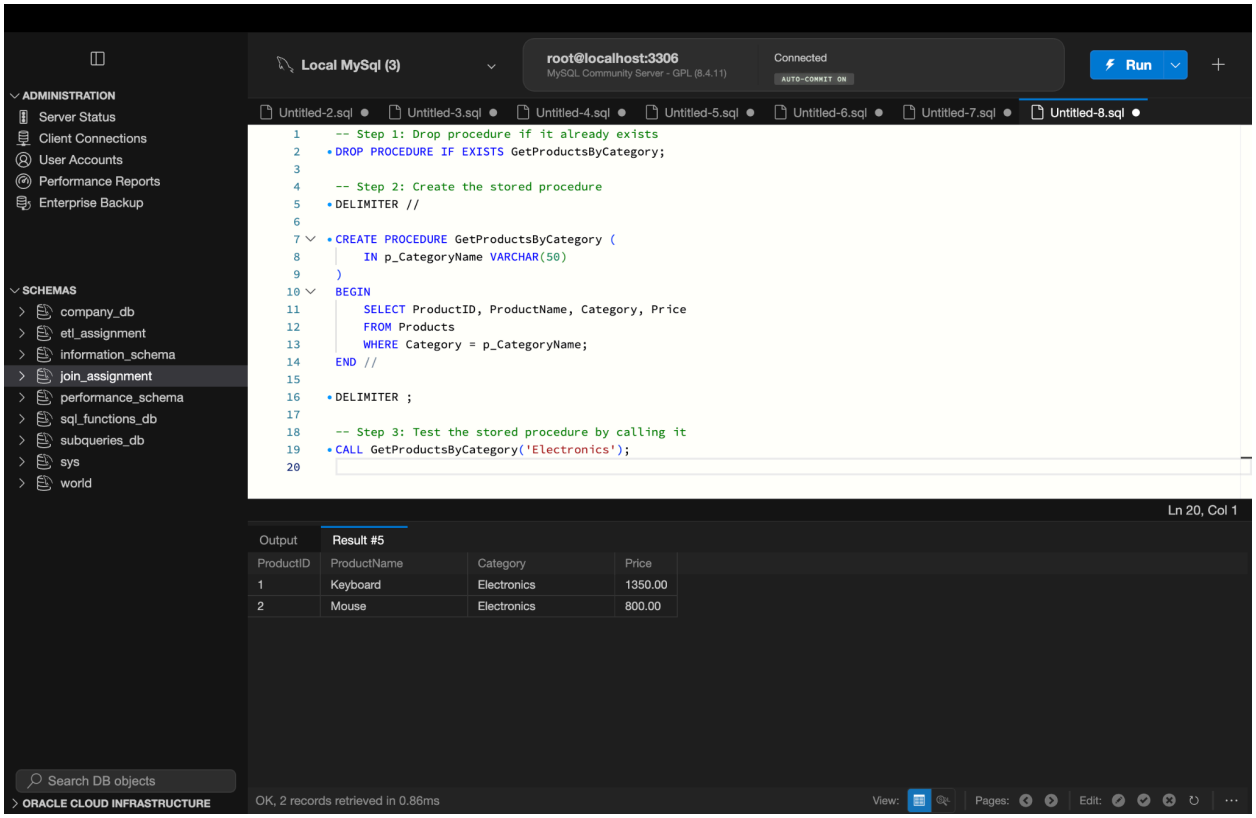

Local MySql (3)
root@localhost:3306
MySQL Community Server - GPL (8.4.11)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
company_db
etl_assignment
information_schema
join_assignment
performance_schema
sql_functions_db
subqueries_db
sys
world
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled-2.sql
Untitled-3.sql
Untitled-4.sql
Untitled-5.sql
Untitled-6.sql
1 •CREATE VIEW vw_CategorySummary AS
2 SELECT
3 Category,
4 COUNT(ProductID) AS TotalProducts,
5 AVG(Price) AS AveragePrice
6 FROM Products
7 GROUP BY Category;
8
9 -- To test the view, run:
10 -- SELECT * FROM vw_CategorySummary;
11
Ln 11, Col 1
Output
#1: OK, 0 records retrieved in 2.738ms
#2: OK, 0 records retrieved in 0.412ms
✓ SQL Script execution completed in 65ms. 2 statements executed successfully.


Local MySql (3)
root@localhost:3306
MySQL Community Server - GPL (8.4.11)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
company_db
etl_assignment
information_schema
join_assignment
performance_schema
sql_functions_db
subqueries_db
sys
world
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled-2.sql
Untitled-3.sql
Untitled-4.sql
Untitled-5.sql
Untitled-6.sql
1 •SELECT * FROM vw_CategorySummary;
2
3
4
5
6
Ln 2, Col 1
Output
Result #1
Category
TotalProducts
AveragePrice
Electronics
2
1000.000000
Furniture
2
4000.000000
OK, 2 records retrieved in 2.632ms
View:
Pages:
Edit:

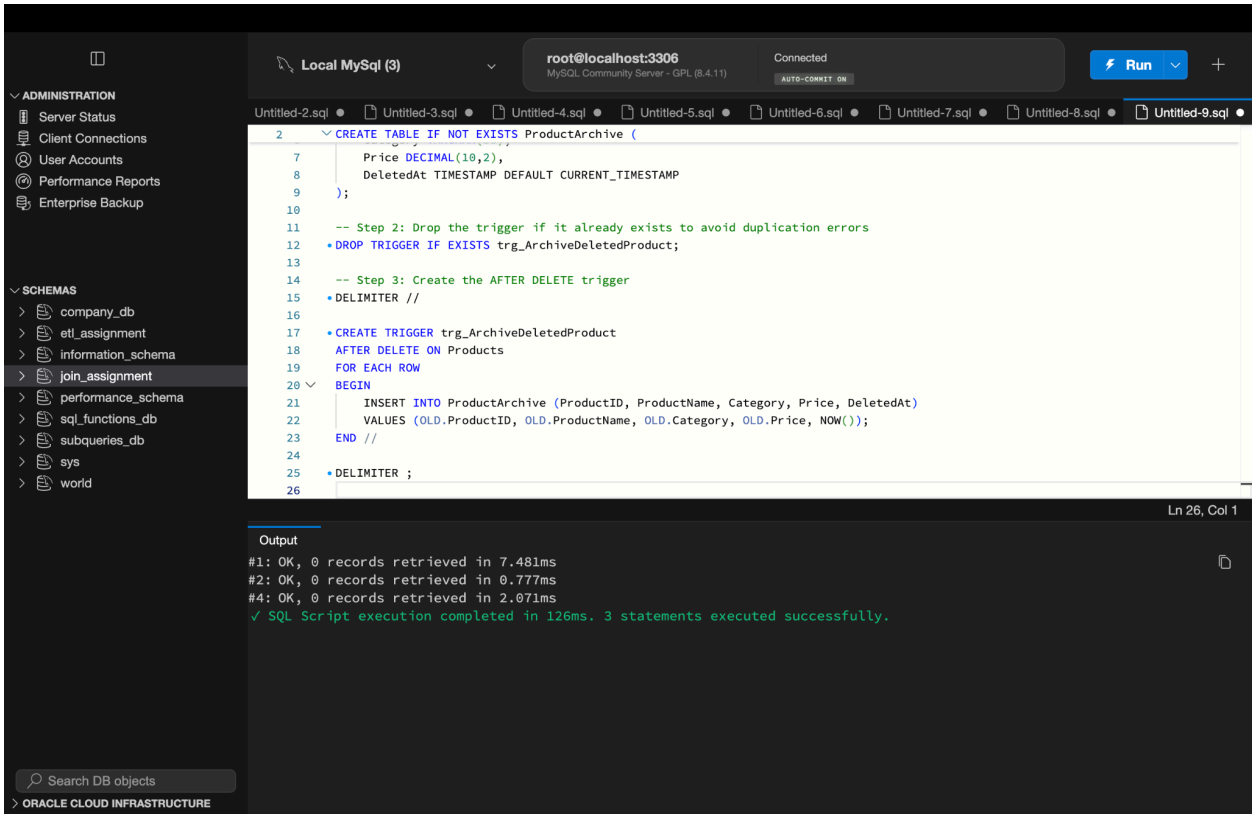
Q8. Create an updatable view containing ProductID, ProductName, and Price. Then update the price of ProductID = 1 using the view.



Q9. Create a stored procedure that accepts a category name and returns all products belonging to that category.



Q10. Create an AFTER DELETE trigger on the table that archives deleted product rows into a new table . The archive should store ProductID, ProductName, Category, Price, and DeletedAt timestamp.


Local MySql (3)
root@localhost:3306
MySQL Community Server - GPL (8.4.11)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
company_db
etl_assignment
information_schema
join_assignment
performance_schema
sql_functions_db
subqueries_db
sys
world
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled-2.sql
Untitled-3.sql
Untitled-4.sql
Untitled-5.sql
Untitled-6.sql
Untitled-7.sql
Untitled-8.sql
Untitled-9.sql
2 CREATE TABLE IF NOT EXISTS ProductArchive (
7 Price DECIMAL(10,2),
8 DeletedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
9);
10
11 -- Step 2: Drop the trigger if it already exists to avoid duplication errors
12 DROP TRIGGER IF EXISTS trg_ArchiveDeletedProduct;
13
14 -- Step 3: Create the AFTER DELETE trigger
15 DELIMITER //
16
17 CREATE TRIGGER trg_ArchiveDeletedProduct
18 AFTER DELETE ON Products
19 FOR EACH ROW
20 BEGIN
21 INSERT INTO ProductArchive (ProductID, ProductName, Category, Price, DeletedAt)
22 VALUES (OLD.ProductID, OLD.ProductName, OLD.Category, OLD.Price, NOW());
23 END //
24
25 DELIMITER ;
26
Ln 26, Col 1
Output
#1: OK, 0 records retrieved in 7.481ms
#2: OK, 0 records retrieved in 0.777ms
#4: OK, 0 records retrieved in 2.071ms
✓ SQL Script execution completed in 126ms. 3 statements executed successfully.